

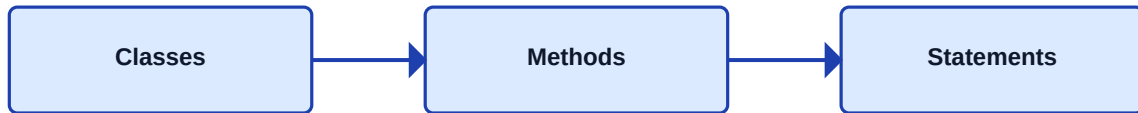
# SOEN 240 – Java Fundamentals

Program Structure • Methods • Variables • Data Types • Scanner | Kisii University | Clean Revision Notes

## 1. Methods & Basic Program Structure

**Methods** are blocks of code that perform a specific task. Every Java **application** must contain a **main** method (the entry point). An **applet** must *not* have a main method.

A Java program is organised hierarchically:



*Basic structure of a Java program (Classes contain Methods; Methods contain Statements)*

### Typical source-file layout

```
// Comments can appear anywhere
public class Welcome {
    // class header → class body
    public static void main(String[] args) {
        // statements ...
    }
}
```

*Comments may appear anywhere; the class body holds fields and methods*

## 2. Access Modifiers

Access modifiers control which other classes can see or use a class, method or variable.

| Modifier             | Meaning (from notes)                                           |
|----------------------|----------------------------------------------------------------|
| <b>public</b>        | Allows other classes to interact with it                       |
| <b>private</b>       | Has some restriction to access (visible only inside the class) |
| default (no keyword) | Package-private – visible only inside the same package         |
| <b>protected</b>     | Visible inside the package and to subclasses                   |

## 3. Anatomy of a Method

A method declaration contains (all optional parts shown):

- **Optional modifiers** (public, static, final, ...)
- **Return type** (or **void** if nothing is returned)
- **Name**
- **Set of arguments** with their types (the parameter list)

You may declare variables inside a class (fields) or inside a method (local variables). Fields exist for the lifetime of the object; local variables exist only while the method runs.

## 4. Variables

**Variable** – a named location in computer memory that holds a value of a certain data type. A variable must be declared before it is used; the declaration allocates memory for that type.

### Declaration syntax

Datatype VariableName; e.g. int total; char grade; long value; int x, y, z;

Variables can be **primitive** (store a single value) or **reference** (store an address of an object).

### Initialisation

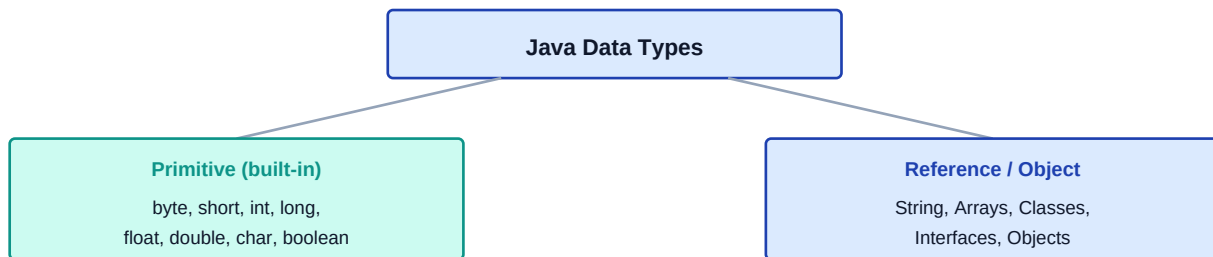
A variable can be given an initial value at declaration using the assignment operator:

```
int sum = 0;  boolean status = false;
```

An **assignment statement** later changes the value: `VariableName = expression;`

## 5. Data Types

A **data type** is a set of values together with the operations that can be performed on those values. Java supplies several predefined (built-in) **primitive** types; you can also create your own types by defining classes.



*Classification of Java data types*

### Primitive types – store a single value; named by a keyword

Integer types (exact whole numbers):

| Type  | Size   | Range (approx.)              |
|-------|--------|------------------------------|
| byte  | 8 bit  | −128 → 127                   |
| short | 16 bit | −32 768 → 32 767             |
| int   | 32 bit | ~ ±2 billion (9 digits)      |
| long  | 64 bit | ~ ±9 quintillion (18 digits) |

Floating-point: **float** (32-bit) and **double** (64-bit).

Character: **char** stores a single character written in single quotes, e.g. 'A'.

Boolean: **boolean** represents a true/false condition (any two-state situation such as On/Off). The only legal values are the reserved words true and false.

### Reference (non-primitive) types

Include Classes, Arrays, Objects and Interfaces. A reference variable may refer to more than one type through inheritance or interfaces. You create your own types by defining a class and then declaring a variable of that class, e.g. HelloWorld a;

**String** – a sequence of characters. In Java a String is an object of the class java.lang.String.

**Array** – a homogeneous collection (all elements the same type). Implemented as objects; elements are accessed by index.

## 6. Constants

A **constant** is an identifier that is similar to a variable except that its value never changes during the program's execution. In Java the reserved word **final** is used to declare a constant:

```
final double PI = 3.142;
```

Any later assignment that tries to put a new value into PI is flagged as a compile-time error. Using constants improves readability and makes maintenance safer (change the value in one place only).

## 7. Identifiers & Whitespace

**Identifiers** are the names a programmer gives to classes, methods, variables, etc.

### Rules

- Combination of letters, digits, underscore ( `_` ) and dollar sign ( `$` )
- Must be a single word (no spaces)
- Unrestricted length
- Case-sensitive
- First character cannot be a digit
- Keywords (reserved words) cannot be used as identifiers

### Sources of identifier names

- Fixed Java reserved words – e.g. public, class, static, void

- Chosen by the programmer – e.g. HelloWorld, Main, args
- Chosen by the author of a library we use – e.g. String, println, out

**Whitespace** (spaces, tabs, blank lines) is used to separate tokens and improve readability. Java treats any amount of consecutive whitespace as a single separator; extra spaces are ignored by the compiler.

## 8. Classes & Interfaces (brief)

**Class** – a blueprint that contains fields (variables) and methods describing the behaviour of an object.

**Interface** – can contain method signatures and variables. Methods are by default abstract (signature only, no body).

## 9. Scanner Class – Reading User Input

The **Scanner** class (from java.util) is used to read input from the user (or from any input stream).

Typical pattern:

```
Scanner read = new Scanner(System.in); // source = keyboard
```

```
System.out.print("Enter the value ");
```

```
int value = read.nextInt(); // or nextDouble(), nextLine(), ...
```

Common methods: nextLine() (whole line as String), nextInt(), nextDouble(), etc. – each reads the next token of the indicated type.

## Quick Revision Checklist

- Every Java application needs a public static void main(String[] args) method; applets do not.
- Structure: Classes contain Methods; Methods contain Statements.
- Access modifiers: public, private, protected, default (package).
- Variable must be declared before use; declaration allocates memory.
- Primitive types store one value; reference types store an object address.
- Integer sizes: byte 8, short 16, int 32, long 64 bits.
- final creates a constant – value cannot be changed later.
- boolean only true / false; char uses single quotes.
- Identifier rules: no spaces, cannot start with digit, not a keyword, case-sensitive.
- Scanner: new Scanner(System.in) then nextInt() / nextDouble() / nextLine().